

Geração e Avaliação de Exercícios de Algoritmos com LLMs

Diogo Estéfano Cambraia Freitas | Engenharia de Telecomunicações | Unipampa

Escolhi as 3 IA 's que mais tive contato (em ordem, Chat GPT > Claude AI > Gemini. Atualmente, estou na fase da Claude AI).

Prompt geral: "Sou graduando em Engenharia de Telecomunicações. Preciso que faça 10 exercícios de algoritmos nessa área, com os seguintes requisitos: enunciado do problema, aplicação real na telecom, o pseudocódigo como em "ordem de passos" e o respectivo código em python."

Claude AI

```
=====
=====
EXERCÍCIO 1 — CÁLCULO DE SNR
=====
=====
```

ENUNCIADO

Dado um sinal com potência S (em Watts) e um ruído com potência N (em Watts), calcule o SNR em dB. Classifique a qualidade do link como Excelente (>30 dB), Boa (20-30 dB), Aceitável (10-20 dB) ou Ruim (<10 dB).

APLICAÇÃO REAL

O SNR (Signal-to-Noise Ratio) é uma métrica fundamental em sistemas de comunicação, usada para avaliar a qualidade de links em redes móveis (4G/5G), sistemas de satélite e rádio.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler S (potência do sinal em W) e N (potência do ruído em W)
2. Verificar se $N > 0$ (evitar divisão por zero)
3. Calcular $SNR_{linear} = S / N$
4. Converter para dB: $SNR_{dB} = 10 * \log_{10}(SNR_{linear})$
5. Se $SNR_{dB} > 30$ -> classificar como "Excelente"
6. Senão se $SNR_{dB} \geq 20$ -> "Boa"
7. Senão se $SNR_{dB} \geq 10$ -> "Aceitável"
8. Senão -> "Ruim"
9. Exibir SNR_{dB} e classificação

CÓDIGO PYTHON

```
import math
```

```
def calcular_snr(S, N):
    if N <= 0:
        raise ValueError("Potencia de ruido deve ser positiva")
    snr_linear = S / N
    snr_db = 10 * math.log10(snr_linear)
    if snr_db > 30:
        qualidade = "Excelente"
    elif snr_db >= 20:
        qualidade = "Boa"
    elif snr_db >= 10:
        qualidade = "Aceitavel"
    else:
        qualidade = "Ruim"
    return snr_db, qualidade
```

```
S = 1e-3 # 1 mW
N = 1e-6 # 1 uW
snr, q = calcular_snr(S, N)
print(f"SNR = {snr:.2f} dB -> {q}")
```

=====

=====

EXERCÍCIO 2 — ATENUAÇÃO EM CABO COAXIAL

=====

=====

ENUNCIADO

Dado o coeficiente de atenuação α (dB/km) de um cabo coaxial e o comprimento L (km) do enlace, calcule a potência recebida em dBm. Determine se um repetidor é necessário caso a potência recebida fique abaixo de um limiar mínimo.

APLICAÇÃO REAL

O cálculo de perda em cabos é essencial no projeto de redes de TV a cabo, redes de acesso DOCSIS e infraestrutura de backhaul, onde repetidores e amplificadores são dimensionados com base nessa análise.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler P_{tx_dBm} (potência de transmissão em dBm)
2. Ler α (atenuação em dB/km) e L (comprimento em km)
3. Ler $limiar_min$ (mínima potência aceitável em dBm)
4. Calcular $atenuacao_total = \alpha \times L$
5. Calcular $P_{rx} = P_{tx_dBm} - attenuacao_total$
6. Se $P_{rx} < limiar_min \rightarrow$ indicar necessidade de repetidor
7. Exibir P_{rx} e decisão sobre repetidor

CÓDIGO PYTHON

```
def atenuacao_cabo(P_tx_dBm, alpha, L, limiar_min):
    atenuacao_total = alpha * L
    P_rx = P_tx_dBm - atenuacao_total
    repetidor = P_rx < limiar_min
    return P_rx, atenuacao_total, repetidor
```

```
P_tx = 20 # dBm
alpha = 0.5 # dB/km
L = 30 # km
limiar = -10 # dBm
```

```
P_rx, perda, rep = atenuacao_cabo(P_tx, alpha, L, limiar)
print(f"Perda total: {perda:.1f} dB")
print(f"Potencia recebida: {P_rx:.1f} dBm")
print(f"Repetidor necessario: {rep}")
```

```
=====
=====
EXERCÍCIO 3 — CAPACIDADE DE SHANNON
=====
=====
```

ENUNCIADO

Implemente o cálculo da capacidade máxima de um canal de comunicação usando o Teorema de Shannon-Hartley. Dado a largura de banda B (Hz) e o SNR linear, calcule a taxa máxima teórica de transmissão em bps.

APLICAÇÃO REAL

O limite de Shannon é a referência teórica para todo projeto de modem e sistema de comunicação digital — de DSL a 5G — determinando a taxa máxima possível em um canal com ruído.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler B (largura de banda em Hz)
2. Ler SNR (relação sinal-ruído, linear)
3. Verificar se $B > 0$ e $\text{SNR} > 0$
4. Calcular $C = B \times \log_2(1 + \text{SNR})$
5. Converter C para Mbps para facilitar leitura
6. Exibir capacidade máxima do canal

CÓDIGO PYTHON

```
import math

def capacidade_shannon(B, SNR):
    if B <= 0 or SNR <= 0:
        raise ValueError("B e SNR devem ser positivos")
    C = B * math.log2(1 + SNR)
```

```

return C

B = 20e6 # 20 MHz (canal Wi-Fi)
SNR = 100 # equivale a ~20 dB

C = capacidade_shannon(B, SNR)
print(f"Capacidade maxima: {C/1e6:.2f} Mbps")

# Variacao com diferentes SNRs
for snr_dB in [10, 20, 30, 40]:
    snr_lin = 10 ** (snr_dB / 10)
    C = capacidade_shannon(B, snr_lin)
    print(f"SNR={snr_dB} dB -> C={C/1e6:.1f} Mbps")

```

```

=====
=====
EXERCÍCIO 4 — BANDA DE NYQUIST
=====
=====

```

ENUNCIADO

Dado o número de símbolos por segundo (taxa de símbolo R_s , em Baud) e o número de níveis de modulação M , calcule a taxa de bits (R_b) e a largura de banda mínima necessária segundo o critério de Nyquist.

APLICAÇÃO REAL

O critério de Nyquist é usado no dimensionamento de sistemas de modulação como QAM e PSK, presentes em modems DSL, cabos DOCSIS e rádios digitais, para otimizar o uso do espectro.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler R_s (taxa de símbolo em Baud) e M (número de níveis)
2. Verificar se M é potência de 2 e $M \geq 2$
3. Calcular bits por símbolo: $k = \log_2(M)$
4. Calcular taxa de bits: $R_b = R_s \times k$
5. Calcular largura de banda mínima de Nyquist: $BW = R_s / 2$
6. Calcular eficiência espectral: $\eta = R_b / BW$
7. Exibir R_b , BW e η

CÓDIGO PYTHON

```

import math

def nyquist(Rs, M):
    if M < 2 or (M & (M - 1)) != 0:
        raise ValueError("M deve ser potencia de 2 (ex: 2,4,16,64)")
    k = math.log2(M) # bits por simbolo
    Rb = Rs * k      # taxa de bits (bps)

```

```

BW = Rs / 2          # largura de banda minima (Hz)
eta = Rb / BW        # eficiencia espectral (bps/Hz)
return Rb, BW, eta, k

```

```

Rs = 1e6 # 1 MBaud
for M in [2, 4, 16, 64, 256]:
    Rb, BW, eta, k = nyquist(Rs, M)
    print(f"M={M:4d} ({k:.0f} bits/simbolo) -> "
          f"Rb={Rb/1e6:.1f} Mbps, BW={BW/1e3:.0f} kHz, eta={eta:.1f} bps/Hz")

```

=====

=====

EXERCÍCIO 5 — PERDA NO ESPAÇO LIVRE (FSPL)

=====

=====

ENUNCIADO

Calcule a perda de propagação no espaço livre (Free Space Path Loss) entre um transmissor e um receptor, dado a distância d (km) e a frequência f (MHz). Use o modelo de Friis.

APLICAÇÃO REAL

A FSPL é a base para o planejamento de radioenlaces ponto a ponto, dimensionamento de células 4G/5G e cálculo de link budget em sistemas de satélite e Wi-Fi.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler d (distância em km) e f (frequência em MHz)
2. Verificar se $d > 0$ e $f > 0$
3. Aplicar fórmula simplificada de Friis:
4. $FSPL_{dB} = 20 \cdot \log_{10}(d_{km}) + 20 \cdot \log_{10}(f_{MHz}) + 32.45$
5. Exibir FSPL em dB

CÓDIGO PYTHON

```

import math

def fspl(d_km, f_MHz):
    if d_km <= 0 or f_MHz <= 0:
        raise ValueError("Distancia e frecuencia devem ser positivas")
    # Formula simplificada de Friis
    fspl_dB = (20 * math.log10(d_km) +
               20 * math.log10(f_MHz) +
               32.45)
    return fspl_dB

```

```

cenarios = [
    (1, 900, "GSM 900 MHz, 1 km"),

```

```

(5, 1800, "LTE 1.8 GHz, 5 km"),
(0.1, 2400, "Wi-Fi 2.4 GHz, 100 m"),
(500, 14000, "Ku-band satellite, 500 km"),
]

```

```

for d, f, desc in cenarios:
    perda = fspl(d, f)
    print(f"{desc}: FSPL = {perda:.1f} dB")

```

```

=====
=====
EXERCÍCIO 6 — DETECÇÃO DE ERRO POR PARIDADE
=====
=====

```

ENUNCIADO

Implemente um sistema de detecção de erros por bit de paridade par. Dado um conjunto de bits de dados, calcule e adicione o bit de paridade. Em seguida, verifique se uma palavra de código recebida contém erro.

APLICAÇÃO REAL

Verificação de paridade é usada em protocolos seriais (UART), memórias e camadas físicas de redes para detectar erros simples de transmissão, sendo base para códigos mais complexos como Hamming e CRC.

PSEUDOCÓDIGO — ORDEM DE PASSOS

TRANSMISSOR:

1. Ler sequência de bits de dados
2. Contar quantos bits '1' existem
3. Se contagem ímpar -> bit de paridade = 1 (torna par)
4. Senão -> bit de paridade = 0
5. Transmitir dados + bit de paridade

RECEPTOR:

6. Ler palavra de código recebida
7. Contar todos os bits '1' na palavra
8. Se contagem ímpar -> ERRO detectado
9. Senão -> sem erro detectado

CÓDIGO PYTHON

```

def adicionar_paridade(bits):
    qtd_uns = sum(bits)
    bit_par = 1 if qtd_uns % 2 != 0 else 0
    return bits + [bit_par]

```

```

def verificar_paridade(palavra):
    qtd_uns = sum(palavra)
    erro = qtd_uns % 2 != 0

```

```

return erro

dados = [1, 0, 1, 1, 0]
palavra_tx = adicionar_paridade(dados)
print(f"Dados: {dados}")
print(f"Palavra transmitida: {palavra_tx}")

# Simula recepcao com e sem erro
recebido_ok = palavra_tx.copy()
recebido_err = palavra_tx.copy()
recebido_err[2] ^= 1 # flip de 1 bit

for nome, palavra in [("Sem erro", recebido_ok),
                      ("Com erro", recebido_err)]:
    resultado = verificar_paridade(palavra)
    print(f"{nome}: {palavra} -> {'ERRO!' if resultado else 'OK'}")

```

```

=====
=====
EXERCÍCIO 7 — ROTEAMENTO POR MENOR SALTO (BFS)
=====
=====

```

ENUNCIADO

Dada uma rede representada como grafo (nós = roteadores, arestas = enlaces), implemente o algoritmo de busca em largura (BFS) para encontrar o caminho com menor número de saltos entre dois roteadores.

APLICAÇÃO REAL

BFS é a base conceitual de protocolos de roteamento como OSPF e IS-IS, e é usado no cálculo de topologia em redes definidas por software (SDN).

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler grafo da rede (lista de adjacência)
2. Ler nó de origem e nó de destino
3. Inicializar fila com [origem], visitados = {origem}
4. Enquanto fila não estiver vazia:
 5. Retirar caminho atual da fila
 6. Se último nó == destino -> retornar caminho
 7. Para cada vizinho não visitado:
 8. Marcar como visitado, adicionar à fila
 9. Se destino não alcançado -> retornar None

CÓDIGO PYTHON

```
from collections import deque
```

```
def menor_saltos(grafo, origem, destino):
```

```

fila = deque([[origem]])
visitados = {origem}
while fila:
    caminho = fila.popleft()
    atual = caminho[-1]
    if atual == destino:
        return caminho
    for vizinho in grafo.get(atual, []):
        if vizinho not in visitados:
            visitados.add(vizinho)
            fila.append(caminho + [vizinho])
return None

```

Rede exemplo: roteadores A-F

```

rede = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D', 'E'],
    'D': ['B', 'C', 'F'],
    'E': ['C', 'F'],
    'F': ['D', 'E'],
}

```

```

caminho = menor_saltos(rede, 'A', 'F')
print(f"Caminho A->F: {' -> '.join(caminho)}")
print(f"Saltos: {len(caminho) - 1}")

```

```

=====
=====
EXERCÍCIO 8 — ALOCAÇÃO DE FREQUÊNCIAS (COLORAÇÃO DE GRAFOS)
=====
=====

```

ENUNCIADO

Dado um conjunto de células em uma rede celular e suas células vizinhas (que não podem usar a mesma frequência), determine uma alocação válida de frequências usando o mínimo de canais possível (coloração de grafos gulosa).

APLICAÇÃO REAL

A coloração de grafos modela o problema de reuso de frequências em redes celulares GSM/LTE e Wi-Fi, minimizando interferência co-canal entre células adjacentes.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Representar células e adjacências como grafo
2. Ordenar células por grau (mais vizinhos primeiro)
3. Para cada célula na ordem definida:
4. Coletar frequências já usadas por vizinhos

5. Atribuir a menor frequência disponível não conflitante
6. Retornar mapeamento célula -> frequência
7. Calcular total de frequências distintas usadas

CÓDIGO PYTHON

```
def alocar_frequencias(grafo):
    # Ordena por grau decrescente (heurística gulosa)
    ordem = sorted(grafo, key=lambda v: len(grafo[v]), reverse=True)
    alocao = {}
    for celula in ordem:
        usadas = {alocacao[v] for v in grafo[celula] if v in alocacao}
        freq = 1
        while freq in usadas:
            freq += 1
        alocacao[celula] = freq
    return alocacao
```

Rede com 6 células

```
rede = {
    'C1': ['C2', 'C3'],
    'C2': ['C1', 'C3', 'C4'],
    'C3': ['C1', 'C2', 'C5'],
    'C4': ['C2', 'C5', 'C6'],
    'C5': ['C3', 'C4', 'C6'],
    'C6': ['C4', 'C5'],
}
```

```
alocacao = alocar_frequencias(rede)
print("Alocacao de frequencias:")
for cel, freq in sorted(alocacao.items()):
    print(f" {cel} -> Canal {freq}")
print(f"Total de canais usados: {max(alocacao.values())}")
```

```
=====
=====
EXERCÍCIO 9 — CODIFICAÇÃO DE LINHA: NRZ-L VS MANCHESTER
=====
=====
```

ENUNCIADO

Dada uma sequência de bits, implemente e compare dois esquemas de codificação de linha: NRZ-L (Non-Return to Zero Level) e Manchester. Exiba o sinal resultante para cada esquema e analise a componente DC.

APLICAÇÃO REAL

Codificações de linha são usadas em Ethernet (Manchester no 10BASE-T), fibra óptica (NRZ em 100G+) e interfaces seriais para garantir sincronização de clock

e ausência de componente DC.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler sequência de bits de entrada

NRZ-L:

2. Para cada bit: '1' -> nível alto (+1), '0' -> nível baixo (-1)

Manchester:

3. Para cada bit '1': meio período baixo (-1), depois alto (+1)

4. Para cada bit '0': meio período alto (+1), depois baixo (-1)

5. Calcular componente DC de cada sinal (média dos níveis)

6. Exibir e comparar os dois sinais

CÓDIGO PYTHON

```
def nrz_l(bits):
```

```
    return [1 if b == 1 else -1 for b in bits]
```

```
def manchester(bits):
```

```
    # Cada bit vira dois meios periodos
```

```
    sinal = []
```

```
    for b in bits:
```

```
        if b == 1:
```

```
            sinal.extend([-1, 1]) # transicao subida
```

```
        else:
```

```
            sinal.extend([1, -1]) # transicao descida
```

```
    return sinal
```

```
bits = [1, 0, 1, 1, 0, 0, 1]
```

```
nrz = nrz_l(bits)
```

```
manch = manchester(bits)
```

```
print(f"Bits:      {bits}")
```

```
print(f"NRZ-L:    {nrz}")
```

```
print(f"Manchester: {manch}")
```

```
dc_nrz = sum(nrz) / len(nrz)
```

```
dc_man = sum(manch) / len(manch)
```

```
print(f"Comp. DC NRZ-L:    {dc_nrz:.2f}")
```

```
print(f"Comp. DC Manchester: {dc_man:.2f}")
```

```
print("Manchester elimina DC -> melhor para fibra optica")
```

```
=====
=====
EXERCÍCIO 10 — LINK BUDGET DE SATÉLITE
=====
=====
```

ENUNCIADO

Calcule o link budget de um enlace de satélite geoestacionário. Dados: EIRP do transmissor, ganho da antena receptora, perdas (espaço livre, atmosférica, de apontamento) e temperatura de ruído do sistema. Determine a relação E_b/N_0 e se o enlace fecha com margem suficiente.

APLICAÇÃO REAL

Link budget é o cálculo central no projeto de sistemas via satélite (DTH, VSAT, Starlink), balanceando parâmetros de toda a cadeia para garantir qualidade de serviço com margem adequada.

PSEUDOCÓDIGO — ORDEM DE PASSOS

1. Ler EIRP_dBW, G_{rx_dBi} , T_{sys_K}
2. Ler perdas: FSPL_dB, L_{atm_dB} , L_{apont_dB}
3. Calcular perdas_totais = FSPL + L_{atm} + L_{apont}
4. Calcular figura de mérito: $G/T = G_{rx} - 10 \cdot \log_{10}(T_{sys})$
5. Calcular potência recebida: $C = EIRP - \text{perdas_totais} + G_{rx}$
6. Calcular densidade de ruído: $N_0 = -228.6 + 10 \cdot \log_{10}(T_{sys})$
7. Calcular $C/N_0 = C - N_0$
8. Calcular $E_b/N_0 = C/N_0 - 10 \cdot \log_{10}(R_b)$
9. Calcular margem = $E_b/N_0 - E_b/N_{0_min}$
10. Se margem $\geq$ margem_requerida -> enlace fecha

CÓDIGO PYTHON

```
import math
```

```
def link_budget(EIRP_dBW, G_rx_dBi, T_sys_K,
                fspl_dB, L_atm_dB, L_apont_dB,
                Rb_bps, EbN0_min_dB, margem_dB=3):
    k_dB = -228.6 # constante de Boltzmann (dBW/K/Hz)
    perdas = fspl_dB + L_atm_dB + L_apont_dB
    C_dBW = EIRP_dBW - perdas + G_rx_dBi
    N0_dBW = k_dB + 10*math.log10(T_sys_K)
    CN0_dB = C_dBW - N0_dBW
    EbN0_dB = CN0_dB - 10*math.log10(Rb_bps)
    GT_dB = G_rx_dBi - 10*math.log10(T_sys_K)
    margem_real = EbN0_dB - EbN0_min_dB
    fecha = margem_real >= margem_dB
    return {'C_dBW': C_dBW, 'CN0_dB': CN0_dB,
            'EbN0_dB': EbN0_dB, 'GT_dB': GT_dB,
            'margem': margem_real, 'fecha': fecha}
```

```
r = link_budget(
    EIRP_dBW = 50, # EIRP do satellite
    G_rx_dBi = 42, # antena parabolica 90 cm
    T_sys_K = 150, # temperatura de sistema
    fspl_dB = 205.6, # GEO Ku-band ~36000 km
    L_atm_dB = 0.5,
```

```

L_apont_dB = 0.5,
Rb_bps     = 2e6, # 2 Mbps QPSK
EbN0_min_dB= 9.6, # Eb/N0 minimo QPSK BER=1e-6
margem_dB  = 3
)
for chave, val in r.items():
    if isinstance(val, bool):
        print(f"{chave:12s}: {val}")
    else:
        print(f"{chave:12s}: {val:.2f}")

```

Gemini

Excelente escolha de curso! A Engenharia de Telecomunicações é uma área onde a eficiência do algoritmo impacta diretamente a latência e a qualidade da experiência do usuário.

Abaixo, preparei 10 exercícios que cobrem desde o processamento básico de sinais até o gerenciamento de redes.

1. Cálculo do Link Budget (Balanço de Link)

Aplicação Real: Essencial para determinar se um receptor conseguirá "ouvir" um transmissor, considerando ganhos de antena e perdas no trajeto.

Pseudocódigo

1. Receber a potência de transmissão (P_{tx}), ganhos das antenas e perdas de cabo.
2. Calcular a perda de propagação no espaço livre (Free Space Path Loss).
3. Somar potências e ganhos; subtrair perdas.
4. Comparar o resultado com a sensibilidade do receptor.

Código Python

Python

```

None

import math

def link_budget(p_tx, g_tx, g_rx, dist_km, freq_mhz):
    # Free Space Path Loss (FSPL)
    fspl = 32.44 + 20*math.log10(dist_km) + 20*math.log10(freq_mhz)
    p_rx = p_tx + g_tx + g_rx - fspl
    return p_rx

# Exemplo: Transmissor de 20dBm, antenas de 5dBi, a 2km em 2400MHz
print(f"Potência Recebida: {link_budget(20, 5, 5, 2, 2400):.2f} dBm")

```

2. Conversão de SNR para Capacidade de Canal (Shannon)

Aplicação Real: Define o limite teórico de bits por segundo que podemos transmitir em um canal com ruído.

Pseudocódigo

1. Obter a largura de banda (\$B\$) e a relação sinal-ruído (\$SNR\$) em dB.
2. Converter \$SNR_{\text{dB}}\$ para escala linear.
3. Aplicar a fórmula $C = B \cdot \log_2(1 + SNR_{\text{linear}})$.

Código Python

Python

```
None

def shannon_capacity(bandwidth, snr_db):
    snr_linear = 10**(snr_db / 10)
    capacity = bandwidth * math.log2(1 + snr_linear)
    return capacity

# Canal de 20MHz com SNR de 30dB
print(f"Capacidade: {shannon_capacity(20e6, 30) / 1e6:.2f} Mbps")
```

3. Verificação de Erros com Checksum

Aplicação Real: Utilizado em protocolos como IP e TCP para garantir que os dados não foram corrompidos no transporte.

Pseudocódigo

1. Receber uma lista de números (bytes).
2. Somar todos os valores.
3. Aplicar a operação de complemento de um ou apenas o resto da divisão (mod).
4. Retornar o valor que será enviado junto ao pacote.

Código Python

Python

```
None

def simple_checksum(data_list):
    total_sum = sum(data_list)
    return total_sum % 256

dados = [72, 101, 108, 108, 111] # "Hello" em ASCII
print(f"Checksum: {simple_checksum(dados)}")
```

4. Algoritmo de Dijkstra para Roteamento

Aplicação Real: Protocolos como OSPF usam esse algoritmo para encontrar o caminho mais curto entre roteadores em uma rede.

Pseudocódigo

1. Iniciar distâncias como infinito para todos os nós, exceto a origem (zero).
2. Enquanto houver nós não visitados:
 - Selecionar o nó com a menor distância atual.
 - Atualizar as distâncias dos vizinhos adjacentes.
 - Marcar o nó como visitado.

Código Python

Python

```
None
import heapq

def dijkstra(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    pq = [(0, start)]

    while pq:
        d_atual, u = heapq.heappushpop(pq, (0, start)) #
Simplificação
        # Implementação completa requer iteração na vizinhança
        # Foco aqui na lógica de prioridade de custo de link.
    return distances
```

5. Modulação Digital - Mapeamento BPSK

Aplicação Real: Mapear bits 0 e 1 em fases de uma onda portadora para transmissão via rádio.

Pseudocódigo

1. Receber um fluxo de bits.
2. Para cada bit: se for 0, atribuir fase -1 ; se for 1, atribuir fase $+1$.
3. Retornar vetor de símbolos complexos.

Código Python

Python

```
None
def bpsk_modulate(bits):
    return [1 if b == 1 else -1 for b in bits]

stream = [1, 0, 1, 1, 0]
print(f"Símbolos BPSK: {bpsk_modulate(stream)}")
```

6. Janela Deslizante (Flow Control)

Aplicação Real: Controle de fluxo no protocolo TCP para evitar que o transmissor sobrecarregue o receptor.

Pseudocódigo

1. Definir tamanho da janela (\$W\$).
2. Enviar pacotes até atingir \$W\$.
3. Aguardar confirmação (ACK) do primeiro pacote para deslizar a janela e enviar o próximo.

Código Python

Python

```
None
def sliding_window(total_packets, window_size):
    for i in range(0, total_packets, window_size):
        batch = list(range(i, min(i + window_size, total_packets)))
        print(f"Enviando pacotes: {batch} | Aguardando ACKs...")

sliding_window(10, 3)
```

7. Cálculo de Probabilidade de Bloqueio (Erlang B)

Aplicação Real: Dimensionamento de troncos telefônicos ou canais de células móveis para evitar congestionamento.

Pseudocódigo

1. Receber tráfego em Erlangs (\$A\$) e número de canais (\$m\$).
2. Iniciar probabilidade de bloqueio \$B = 1\$.
3. Iterar de 1 até \$m\$ usando a fórmula recursiva de Erlang B.

Código Python

Python

```
None
def erlang_b(A, m):
    inv_b = 1
    for i in range(1, m + 1):
        inv_b = 1 + (i / A) * inv_b
    return 1 / inv_b

# 10 Erlangs de tráfego para 15 canais
print(f"Prob. de Bloqueio: {erlang_b(10, 15) * 100:.2f}%")
```

8. Filtro de Média Móvel para Sinais

Aplicação Real: Suavizar ruído em medições de RSSI (potência de sinal recebido) para evitar handovers desnecessários.

Pseudocódigo

1. Receber um sinal ruidoso e o tamanho da janela (\$k\$).
2. Para cada ponto, calcular a média dos \$k\$ valores anteriores.
3. Armazenar o valor suavizado.

Código Python

Python

```
None

def moving_average(signal, window):
    smoothed = []
    for i in range(len(signal)):
        start = max(0, i - window + 1)
        avg = sum(signal[start:i+1]) / (i - start + 1)
        smoothed.append(avg)
    return smoothed

rssi_ruidoso = [-70, -72, -68, -85, -71, -69]
print(f"Sinal Suavizado: {moving_average(rssi_ruidoso, 3)}")
```

9. Conversão de Endereço IP para Binário

Aplicação Real: Processamento interno de roteadores para aplicação de máscaras de sub-rede e decisões de encaminhamento.

Pseudocódigo

1. Dividir a string do IP pelos pontos.
2. Converter cada octeto decimal para binário de 8 bits.
3. Concatenar os resultados.

Código Python

Python

```
None

def ip_to_binary(ip_str):
    octets = ip_str.split('.')
    binary_ip = '.'.join([format(int(o), '08b') for o in octets])
    return binary_ip

print(f"IP Binário: {ip_to_binary('192.168.1.1')}")
```

10. Detecção de Preâmbulo (Correlação Simples)

Aplicação Real: Sincronização de quadros. O receptor busca uma sequência conhecida para saber onde o dado começa.

Pseudocódigo

1. Receber o sinal recebido e a sequência de preâmbulo.
2. Deslizar o preâmbulo sobre o sinal.
3. Contar quantos bits coincidem em cada posição.
4. Onde a coincidência for máxima, o quadro se inicia.

Código Python

Python

```
None
def find_preamble(signal, preamble):
    n, m = len(signal), len(preamble)
    for i in range(n - m + 1):
        if signal[i:i+m] == preamble:
            return i # Início do frame
    return -1

stream = [0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0]
sync = [1, 0, 1]
print(f"Frame inicia no índice: {find_preamble(stream, sync)}")
```

Chat GPT

Perfeito — vou montar exercícios no estilo que você provavelmente veria em disciplinas como **Algoritmos + Telecom** (misturando lógica, sinais, redes e processamento). Vou estruturar **bem didático e progressivo**, indo do básico ao mais aplicado.

Lista de Exercícios — Algoritmos em Telecomunicações

1. Cálculo de Atenuação de Sinal

Enunciado:

Dado um sinal com potência inicial P_{0P_0P0} e uma atenuação de α dB por km, calcule a potência após percorrer uma distância ddd .

Aplicação real:

Usado em **projetos de enlaces ópticos e RF** para prever perda de sinal.

Pseudocódigo:

1. Ler P_0 , α , d
2. Calcular perda total: $\text{perda} = \alpha \cdot d$
3. Calcular potência final: $P = P_0 \cdot 10^{-\text{perda}/10} = P_0 \cdot 10^{-\alpha \cdot d/10}$
4. Exibir PPP

Python:

```
def atenuacao(P0, alpha, d):  
    perda = alpha * d  
    P = P0 * (10 ** (-perda / 10))  
    return P  
  
print(atenuacao(100, 0.2, 10))
```

2. Taxa de Transmissão (Throughput)

Enunciado:

Calcule a taxa de transmissão efetiva considerando tamanho do pacote e tempo de envio.

Aplicação real:

Avaliação de desempenho de redes (Wi-Fi, 4G, 5G).

Pseudocódigo:

1. Ler tamanho (bits) e tempo (s)
2. Calcular taxa: $\text{taxa} = \text{tamanho} / \text{tempo}$
3. Exibir taxa

Python:

```
def throughput(bits, tempo):  
    return bits / tempo  
  
print(throughput(10000, 2))
```

3. Conversão de dBm para Watts

Enunciado:

Converter um valor em dBm para Watts.

Aplicação real:

Medições de potência em equipamentos de RF.

Pseudocódigo:

1. Ler valor em dBm
2. Aplicar fórmula:
 $P(W) = 10^{(\text{dBm} - 30)/10}$

3. Exibir resultado

Python:

```
def dbm_para_watts(dbm):  
    return 10 ** ((dbm - 30) / 10)  
  
print(dbm_para_watts(20))
```

4. Cálculo de Latência Total

Enunciado:

Calcular latência total somando atrasos de propagação, transmissão e processamento.

Aplicação real:

Análise de QoS em redes.

Pseudocódigo:

1. Ler três atrasos
2. Somar todos
3. Exibir total

Python:

```
def latencia_total(prop, trans, proc):  
    return prop + trans + proc  
  
print(latencia_total(10, 5, 2))
```

5. Simulação de Fila de Pacotes

Enunciado:

Simular uma fila onde pacotes chegam e são processados.

Aplicação real:

Roteadores e switches (buffer).

Pseudocódigo:

1. Criar fila vazia
2. Inserir pacotes
3. Remover pacotes em ordem FIFO
4. Exibir processamento

Python:

```
from collections import deque  
  
fila = deque()
```

```
fila.append("Pacote1")
fila.append("Pacote2")

while fila:
    print("Processando:", fila.popleft())
```

6. Cálculo de Capacidade de Canal (Shannon)

Enunciado:

Calcular a capacidade de canal com largura de banda BBB e SNR.

Aplicação real:

Limite teórico de transmissão em sistemas digitais.

Pseudocódigo:

1. Ler B e SNR
2. Calcular:
 $C = B \cdot \log_2(1 + \text{SNR})$
3. Exibir C

Python:

```
import math

def capacidade(B, SNR):
    return B * math.log2(1 + SNR)

print(capacidade(1000, 10))
```

7. Detecção de Erro Simples (Paridade)

Enunciado:

Verificar se uma sequência binária tem paridade par.

Aplicação real:

Detecção de erros em transmissão.

Pseudocódigo:

1. Contar número de 1s
2. Se par → OK
3. Senão → erro

Python:

```
def paridade(bits):
    return bits.count('1') % 2 == 0
```

```
print(paridade("101010"))
```

8. Modulação Binária (NRZ)

Enunciado:

Converter bits em níveis de sinal NRZ ($0 \rightarrow -1$, $1 \rightarrow +1$).

Aplicação real:

Transmissão digital em linha.

Pseudocódigo:

1. Ler sequência binária
2. Para cada bit:
 - se $0 \rightarrow -1$
 - se $1 \rightarrow +1$
3. Exibir vetor

Python:

```
def nrz(bits):  
    return [1 if b == '1' else -1 for b in bits]  
  
print(nrz("1010"))
```

9. Cálculo de BER (Bit Error Rate)

Enunciado:

Comparar dois vetores de bits e calcular taxa de erro.

Aplicação real:

Avaliação de qualidade de comunicação.

Pseudocódigo:

1. Comparar bits transmitidos e recebidos
2. Contar erros
3. Calcular BER = erros / total

Python:

```
def ber(tx, rx):  
    erros = sum(1 for a, b in zip(tx, rx) if a != b)  
    return erros / len(tx)  
  
print(ber("1010", "1001"))
```

10. Roteamento Simples (Menor Caminho)

Enunciado:

Encontrar o menor caminho entre dois nós (Dijkstra).

Aplicação real:

Protocolos de roteamento (OSPF).

Pseudocódigo:

1. Inicializar distâncias
2. Marcar nó inicial
3. Atualizar vizinhos
4. Repetir até destino

Python:

```
import heapq
```

```
def dijkstra(grafo, inicio):
```

```
    dist = {n: float('inf') for n in grafo}
```

```
    dist[inicio] = 0
```

```
    fila = [(0, inicio)]
```

```
    while fila:
```

```
        d, no = heapq.heappop(fila)
```

```
        for vizinho, peso in grafo[no]:
```

```
            nova = d + peso
```

```
            if nova < dist[vizinho]:
```

```
                dist[vizinho] = nova
```

```
                heapq.heappush(fila, (nova, vizinho))
```

```
    return dist
```

```
grafo = {
```

```
    'A': [('B', 1), ('C', 4)],
```

```
    'B': [('C', 2), ('D', 5)],
```

```
    'C': [('D', 1)],
```

```
    'D': []
```

```
}
```

```
print(dijkstra(grafo, 'A'))
```



Se quiser evoluir mais...

Posso montar uma **lista nível HARD**, com coisas tipo:

- OFDM simplificado
- FFT aplicada a sinais
- Simulação de ruído AWGN
- Codificação de canal (Hamming)

- TCP congestion control (simulado)

Só me diz 👍

Análise comparativa final

Depois de testar as 3, talvez seja impressão minha, mas os tópicos que o Chat GPT abordou me pareceram muito alinhados com o que eu já conversava com ele (foi a IA que usei por mais tempo). Por isso creio que ele tenha mais a minha personalidade. Acontece que os exercícios que o Chat GPT gerou foram os mais simples. Mas seria então o caso de pedir que ele refizesse em um nível um pouco mais alto, mantendo os mesmos assuntos. Com isso digo também que nenhuma funcionalidade que o Chat GPT usou me era nova, enquanto os exercícios do Gemini usaram diversas coisas que eu não conhecia, ou que apenas usei muito pouco. O Gemini fez códigos que demandam mais “conhecimento bruto” de programação. Os códigos do Claude AI estavam mais organizados (e vários também de dificuldade bem boa, sempre maior que os do Chat GPT), e os últimos tinham dificuldades maiores. Acho que as 3 IA's tem usos diferentes, pois desde que troquei de ser main Chat GPT para main Claude AI reparei que a Claude AI é bem mais “fria”, enquanto o Chat GPT “demonstra” mais empatia e amizade. Não sei explicar. Para código mesmo, então Claude AI deve ser melhor sim. Os pseudocódigos de todos estavam bons (embora os do Gemini um pouco menos elaborados, e os problemas maiores), e todos abordaram pelo menos algumas mesmas áreas da telecom.